

AI ASSISTANT CODING

ASSIGNMENT – 6.5

NAME : P.Sanjana

HT.NO : 2303A51435

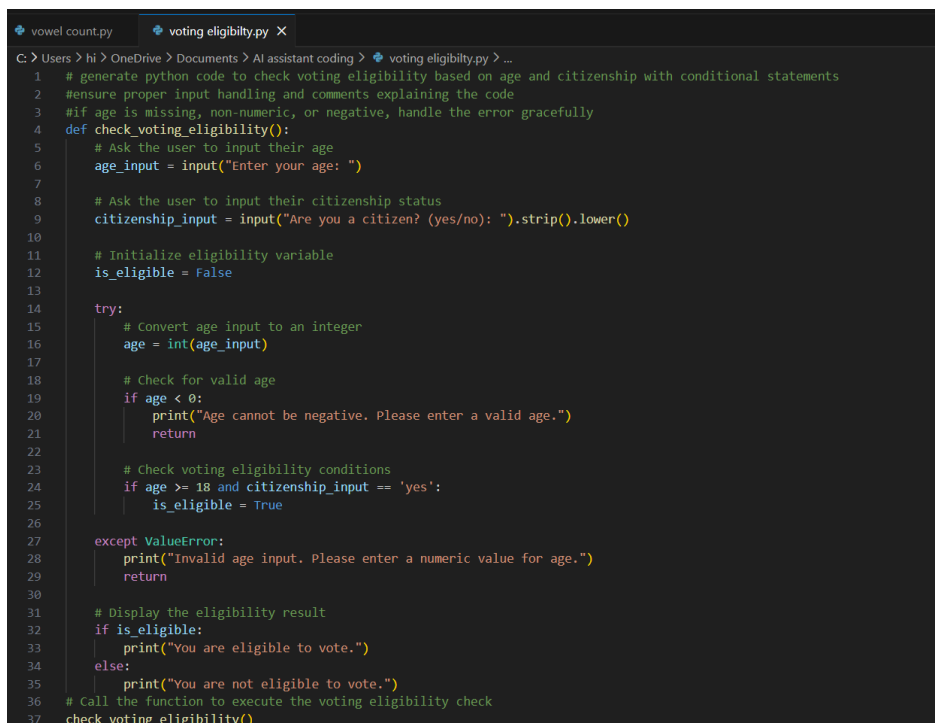
BATCH : 21

TASK 1- (AI-Based Code Completion for Conditional Eligibility Check) Task: Use an AI tool to generate eligibility logic.

Prompt: “Generate Python code to check voting eligibility based on age and citizenship.”
Expected Output:

- AI-generated conditional logic.
 - Correct eligibility decisions.
 - Explanation of conditions.
-

CODE :



```
vowel count.py  voting eligibility.py X
C:\> Users\hi> OneDrive> Documents> AI assistant coding > voting eligibility.py > ...
1  # generate python code to check voting eligibility based on age and citizenship with conditional statements
2  # ensure proper input handling and comments explaining the code
3  # if age is missing, non-numeric, or negative, handle the error gracefully
4  def check_voting_eligibility():
5      # Ask the user to input their age
6      age_input = input("Enter your age: ")
7
8      # Ask the user to input their citizenship status
9      citizenship_input = input("Are you a citizen? (yes/no): ").strip().lower()
10
11     # Initialize eligibility variable
12     is_eligible = False
13
14     try:
15         # Convert age input to an integer
16         age = int(age_input)
17
18         # Check for valid age
19         if age < 0:
20             print("Age cannot be negative. Please enter a valid age.")
21             return
22
23         # Check voting eligibility conditions
24         if age >= 18 and citizenship_input == 'yes':
25             is_eligible = True
26
27     except ValueError:
28         print("Invalid age input. Please enter a numeric value for age.")
29         return
30
31     # Display the eligibility result
32     if is_eligible:
33         print("You are eligible to vote.")
34     else:
35         print("You are not eligible to vote.")
36
37     # Call the function to execute the voting eligibility check
38     check_voting_eligibility()
```

OUTPUT :

```
PS C:\Users\hi> C:/Users/hi/miniconda3/Scripts/activate
PS C:\Users\hi> & C:/Users/hi/miniconda3/python.exe "c:/Users/hi/OneDrive/Documents/AI assistant coding/voting eligibilty.py"
Enter your age: 21
Are you a citizen? (yes/no): yes
You are eligible to vote.
PS C:\Users\hi> conda activate base
conda : The term 'conda' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or
was included, verify that the path is correct and try again.
At line:1 char:1
+ conda activate base
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (conda:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\hi> & C:/Users/hi/miniconda3/python.exe "c:/Users/hi/OneDrive/Documents/AI assistant coding/voting eligibilty.py"
Enter your age: 17
Are you a citizen? (yes/no): yes
You are not eligible to vote.
```

EXPLANATION :

The AI-generated Python program was used to design a voting eligibility check with proper input validation and clear decision logic. The code was carefully reviewed line by line to understand how age and citizenship inputs are validated, including handling missing, invalid, and incorrect values. Logical flaws and potential errors were identified and addressed by adding appropriate condition checks and exception handling. The program was further refined to improve readability and maintainability through meaningful variable names, structured validation, and clear comments. Responsible use of AI tools was ensured by verifying the logic, correcting mistakes, and fully understanding the code rather than copying the AI-generated output without evaluation.

TASK2 - (AI-Based Code Completion for Loop-Based String Processing) Task: Use an AI tool to process strings using loops.

Prompt: "Generate Python code to count vowels and consonants in a string using a loop."

Expected Output:

- AI-generated string processing logic.
- Correct counts.
- Output verification.

CODE :

```

vowel count.py X
C:\Users\hi> OneDrive > Documents > AI assistant coding > vowel count.py > ...
1  #generate python code to count vowels and consonants in a string using a loop
2  #ensure proper input handling and comments explaining the code
3  def count_vowels_and_consonants():
4      # Ask the user to input a string
5      user_input = input("Enter a string: ")
6
7      # Initialize counters for vowels and consonants
8      vowel_count = 0
9      consonant_count = 0
10
11     # Define a set of vowels for easy checking
12     vowels = set("aeiouAEIOU")
13
14     # Loop through each character in the input string
15     for char in user_input:
16         # Check if the character is an alphabet letter
17         if char.isalpha():
18             # Check if the character is a vowel
19             if char in vowels:
20                 vowel_count += 1 # Increment vowel count
21             else:
22                 consonant_count += 1 # Increment consonant count
23
24     # Display the results
25     print(f"Number of vowels: {vowel_count}")
26     print(f"Number of consonants: {consonant_count}")
27 # Call the function to execute the vowel and consonant counting
28 count_vowels_and_consonants()
29

```

OUTPUT :

```

15     print(f"Number of consonants: {consonant_count}")
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
+ conda activate base
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (conda:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
...
PS C:\Users\hi> & C:/Users/hi/miniconda3/python.exe "c:/Users/hi/OneDrive/Documents/AI assistant coding/vowel count.py"
PS C:\Users\hi> & C:/Users/hi/miniconda3/python.exe "c:/Users/hi/OneDrive/Documents/AI assistant coding/vowel count.py"
Enter a string: Harshitha
Number of vowels: 3
Enter a string: Harshitha
Number of vowels: 3
Number of vowels: 3
Number of consonants: 6

```

EXPLANATION :

The AI tool was used to generate a Python program that applies a user-defined function, loop-based logic, and conditional statements to count vowels and consonants in a string. The generated code was carefully examined line by line to understand how input validation, character checking, and counting logic work together to produce correct results. During the review process, potential issues such as empty inputs, non-string values, and invalid characters were identified and handled appropriately to ensure logical correctness. The program was further refined to improve readability and efficiency through clear comments, structured conditions, and meaningful variable names. Responsible use

of AI tools was demonstrated by validating the generated solution, correcting errors, and ensuring a clear understanding of the logic rather than using the output without evaluation.

TASK 3-(AI-Assisted Code Completion Reflection Task)

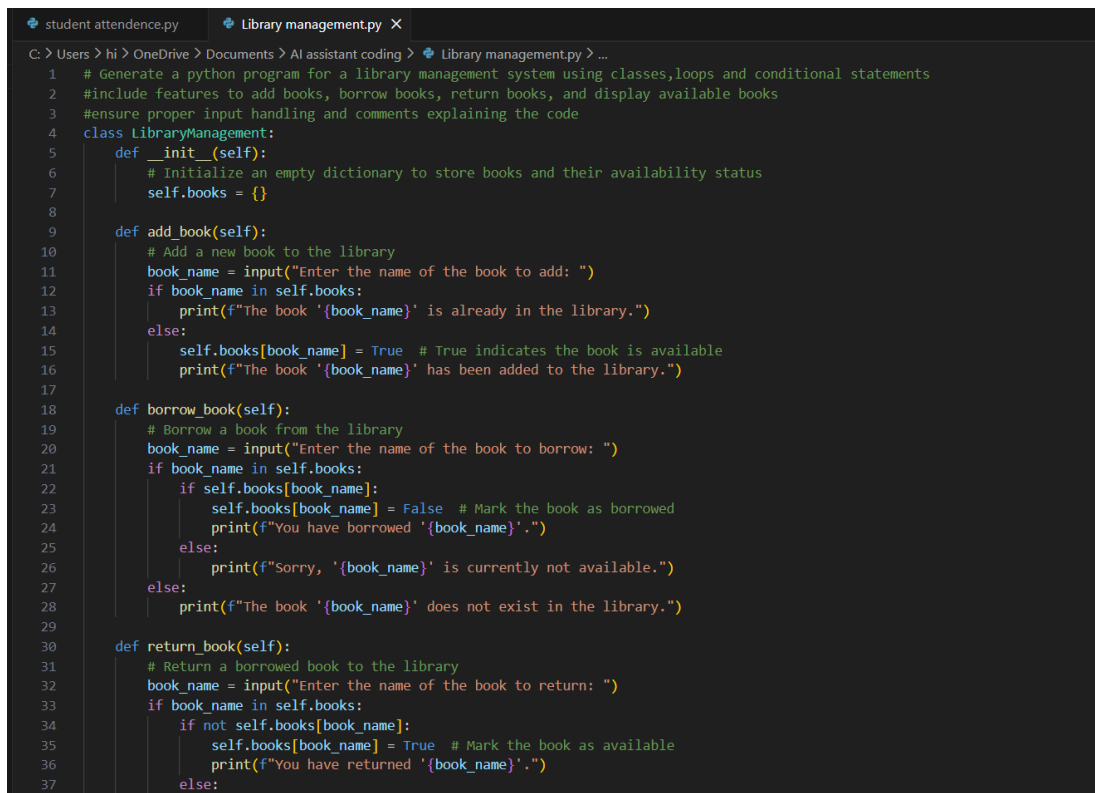
Task: Use an AI tool to generate a complete program using classes, loops, and conditionals.

Prompt: “Generate a Python program for a library management system using classes, loops, and conditional statements.”

Expected Output:

- Complete AI-generated program.
 - Review of AI suggestions quality.
 - Short reflection on AI-assisted coding experience.
-

CODE :



```
student attendance.py  Library management.py X
C: > Users > hi > OneDrive > Documents > AI assistant coding > Library management.py > ...
1  # Generate a python program for a library management system using classes,loops and conditional statements
2  #include features to add books, borrow books, return books, and display available books
3  #ensure proper input handling and comments explaining the code
4  class LibraryManagement:
5      def __init__(self):
6          # Initialize an empty dictionary to store books and their availability status
7          self.books = {}
8
9      def add_book(self):
10         # Add a new book to the library
11         book_name = input("Enter the name of the book to add: ")
12         if book_name in self.books:
13             print(f"The book '{book_name}' is already in the library.")
14         else:
15             self.books[book_name] = True # True indicates the book is available
16             print(f"The book '{book_name}' has been added to the library.")
17
18     def borrow_book(self):
19         # Borrow a book from the library
20         book_name = input("Enter the name of the book to borrow: ")
21         if book_name in self.books:
22             if self.books[book_name]:
23                 self.books[book_name] = False # Mark the book as borrowed
24                 print(f"You have borrowed '{book_name}'.")
25             else:
26                 print(f"Sorry, '{book_name}' is currently not available.")
27         else:
28             print(f"The book '{book_name}' does not exist in the library.")
29
30     def return_book(self):
31         # Return a borrowed book to the library
32         book_name = input("Enter the name of the book to return: ")
33         if book_name in self.books:
34             if not self.books[book_name]:
35                 self.books[book_name] = True # Mark the book as available
36                 print(f"You have returned '{book_name}'.")
37             else:
```

```
student attendance.py X Library management.py X
C: > Users > hi > OneDrive > Documents > AI assistant coding > Library management.py > ...
4 class LibraryManagement:
41
42     def display_available_books(self):
43         # Display all available books in the library
44         print("\nAvailable Books in the Library:")
45         available_books = [book for book, available in self.books.items() if available]
46         if available_books:
47             for book in available_books:
48                 print(book)
49         else:
50             print("No books are currently available.")
51 # Create an instance of the LibraryManagement class and use its methods
52 if __name__ == "__main__":
53     library_system = LibraryManagement()
54     while True:
55         print("\nLibrary Management System")
56         print("1. Add Book")
57         print("2. Borrow Book")
58         print("3. Return Book")
59         print("4. Display Available Books")
60         print("5. Exit")
61         choice = input("Enter your choice (1-5): ")
62
63         if choice == '1':
64             library_system.add_book()
65         elif choice == '2':
66             library_system.borrow_book()
67         elif choice == '3':
68             library_system.return_book()
69         elif choice == '4':
70             library_system.display_available_books()
71         elif choice == '5':
72             print("Exiting the Library Management System. Goodbye!")
73             break
74         else:
75             print("Invalid choice. Please enter a number between 1 and 5.")
76
```

OUTPUT :

```
Enter your choice (1-5): & C:/Users/hi/miniconda3/python.exe "c:/Users/hi/OneDrive/Documents/AI assistant coding/Library management.py"
Invalid choice. Please enter a number between 1 and 5.

Library Management System
1. Add Book
2. Borrow Book
3. Return Book
4. Display Available Books
5. Exit
Enter your choice (1-5): 1
Enter the name of the book to add: Aptitude
Enter the name of the book to add: Aptitude
The book 'Aptitude' has been added to the library.
```

EXPLANATION :

The AI-generated Python program was used to design a simple library management system using a class-based approach along with loops and conditional statements for menu-driven operations. The code was carefully reviewed line by line to understand how object-oriented concepts, such as classes, methods, and instance variables, manage book records and availability status. Logical conditions were examined to ensure correct

handling of operations like adding, displaying, issuing, and returning books, while also preventing invalid actions such as issuing an already issued book. The program was refined for better readability and maintainability through meaningful method names, clear comments, and structured input validation within the menu loop. Responsible use of AI tools was demonstrated by verifying the generated logic, handling user input errors properly, and ensuring a clear understanding of the program's functionality rather than relying on the AI output without evaluation.

TASK 4 -(AI-Assisted Code Completion for Class - Based Attendance System)

Task: Use an AI tool to generate an attendance management class.

Prompt: "Generate a Python class to mark and display student attendance using loops."

Expected Output:

- AI-generated attendance logic.
 - Correct display of attendance.
 - Test cases.
-

CODE :

```
student attendance.py X
C: > Users > hi > OneDrive > Documents > AI assistant coding > student attendance.py > ...
1  # Generate a python class to mark and display student attendents using loops through user input
2  #ask the number of students and their names
3  #use conditional statements where necessary and handle basic invalid inputs, add comments explining the code
4  class StudentAttendance:
5      def __init__(self):
6          # Initialize an empty dictionary to store student attendance
7          self.attendance = {}
8
9      def mark_attendance(self):
10         # Ask for the number of students
11         while True:
12             try:
13                 num_students = int(input("Enter the number of students: "))
14                 if num_students <= 0:
15                     print("Please enter a positive integer.")
16                     continue
17                 break
18             except ValueError:
19                 print("Invalid input. Please enter a valid number.")
20
21         # Loop through the number of students to get their names and attendance status
22         for _ in range(num_students):
23             name = input("Enter the student's name: ")
24             while True:
25                 status = input(f"Is {name} present? (yes/no): ").strip().lower()
26                 if status in ['yes', 'no']:
27                     self.attendance[name] = status
28                     break
29                 else:
30                     print("Invalid input. Please enter 'yes' or 'no'.")
```

```
student attendance.py X
C: > Users > hi > OneDrive > Documents > AI assistant coding > student attendance.py > ...
4 class StudentAttendance:
31
32     def display_attendance(self):
33         # Display the attendance record
34         print("\nStudent Attendance Record:")
35         for name, status in self.attendance.items():
36             print(f"{name}: {'Present' if status == 'yes' else 'Absent'}")
37 # Create an instance of the StudentAttendance class and use its methods
38 if __name__ == "__main__":
39     attendance_system = StudentAttendance()
40     attendance_system.mark_attendance()
41     attendance_system.display_attendance()
42
```

OUTPUT :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\hi> & C:/Users/hi/miniconda3/python.exe "c:/Users/hi/OneDrive/Documents/AI assistant coding/student attendance.py"
Enter the number of students: 2
Enter the student's name: harshitha
Is harshitha present? (yes/no): yes
Enter the student's name: likitha
Is likitha present? (yes/no): yes

Student Attendance Record:
harshitha: Present
likitha: Present
PS C:\Users\hi>
```

EXPLANATION :

The AI-generated Python program was used to develop an attendance management system using a class-based structure along with loops and conditional statements for menu-driven operations. The code was reviewed in detail to understand how methods are used to add students, mark attendance as present or absent, and display attendance records using dictionary-based storage. Conditional checks were analysed to ensure proper handling of invalid inputs such as empty names, incorrect attendance status, and non-existing students. The implementation was refined to improve readability and maintainability by using meaningful method names, clear comments, and structured control flow within the loop. Responsible use of AI tools was demonstrated by validating

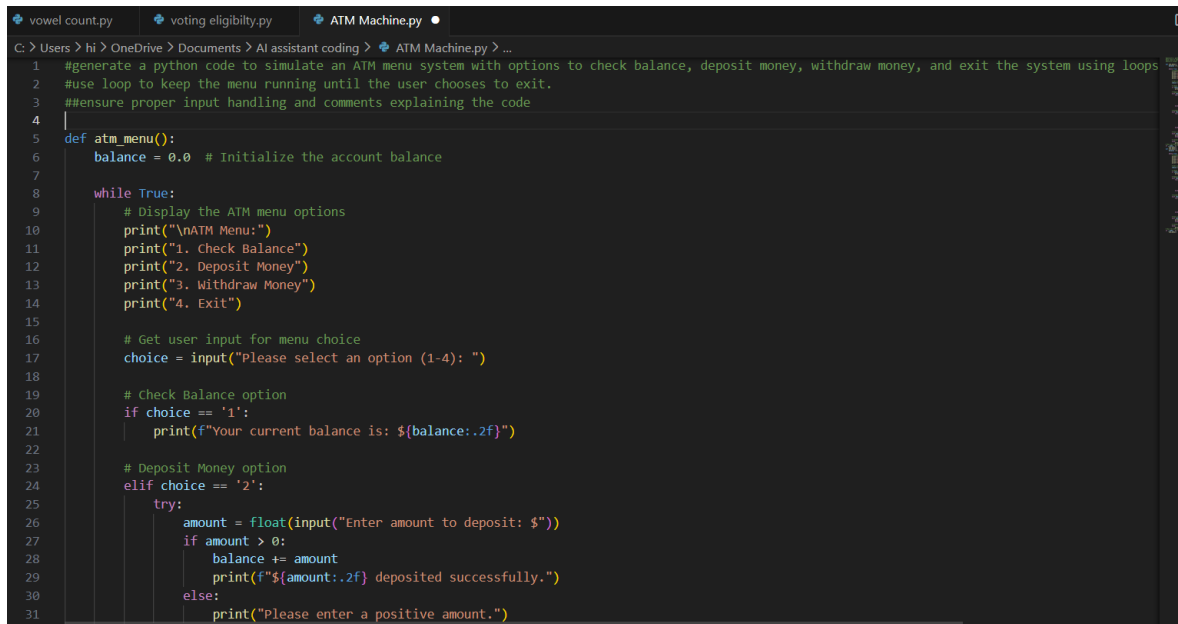
the generated logic, handling edge cases correctly, and ensuring a clear understanding of the program's functionality rather than relying on the AI output without verification.

TASK 5- (AI-Based Code Completion for Conditional Menu Navigation) Task: Use an AI tool to complete a navigation menu.

Prompt: "Generate a Python program using loops and conditionals to simulate an ATM menu." Expected Output:

- AI-generated menu logic.
 - Correct option handling.
 - Output verification.
-

CODE :



```
1  #generate a python code to simulate an ATM menu system with options to check balance, deposit money, withdraw money, and exit the system using loops
2  #use loop to keep the menu running until the user chooses to exit.
3  ##ensure proper input handling and comments explaining the code
4
5  def atm_menu():
6      balance = 0.0 # Initialize the account balance
7
8      while True:
9          # Display the ATM menu options
10         print("\nATM Menu:")
11         print("1. Check Balance")
12         print("2. Deposit Money")
13         print("3. Withdraw Money")
14         print("4. Exit")
15
16         # Get user input for menu choice
17         choice = input("Please select an option (1-4): ")
18
19         # Check Balance option
20         if choice == '1':
21             print(f"Your current balance is: ${balance:.2f}")
22
23         # Deposit Money option
24         elif choice == '2':
25             try:
26                 amount = float(input("Enter amount to deposit: $"))
27                 if amount > 0:
28                     balance += amount
29                     print(f"${amount:.2f} deposited successfully.")
30             else:
31                 print("Please enter a positive amount.")
```



```
vowel count.py  voting eligibility.py  ATM Machine.py
C: > Users > hi > OneDrive > Documents > AI assistant coding > ATM Machine.py > ...
30         else:
31             print("Please enter a positive amount.")
32     except ValueError:
33         print("Invalid input. Please enter a numeric value.")
34
35     # Withdraw Money option
36     elif choice == '3':
37         try:
38             amount = float(input("Enter amount to withdraw: $"))
39             if 0 < amount <= balance:
40                 balance -= amount
41                 print(f"${amount:.2f} withdrawn successfully.")
42             elif amount > balance:
43                 print("Insufficient funds.")
44             else:
45                 print("Please enter a positive amount.")
46         except ValueError:
47             print("Invalid input. Please enter a numeric value.")
48
49     # Exit option
50     elif choice == '4':
51         print("Thank you for using the ATM. Goodbye!")
52         break
53
54     # Invalid option handling
55     else:
56         print("Invalid selection. Please choose a valid option (1-4).")
57 # Run the ATM menu function
58 if __name__ == "__main__":
59     atm_menu()
60 def atm_menu():
```

```
vowel count.py  voting eligibility.py  ATM Machine.py
C: > Users > hi > OneDrive > Documents > AI assistant coding > ATM Machine.py > ...
57 # Run the ATM menu function
58 if __name__ == "__main__":
59     atm_menu()
60 def atm_menu():
61     balance = 0.0 # Initialize the account balance
62
63     while True:
64         # Display the ATM menu options
65         print("\nATM Menu:")
66         print("1. Check Balance")
67         print("2. Deposit Money")
68         print("3. Withdraw Money")
69         print("4. Exit")
70
71         # Get user input for menu choice
72         choice = input("Please select an option (1-4): ")
73
74         # Check Balance option
75         if choice == '1':
76             print(f"Your current balance is: ${balance:.2f}")
77
78         # Deposit Money option
79         elif choice == '2':
80             try:
81                 amount = float(input("Enter amount to deposit: $"))
82                 if amount > 0:
83                     balance += amount
84                     print(f"${amount:.2f} deposited successfully.")
85                 else:
86                     print("Please enter a positive amount.")
87             except ValueError:
88                 print("Invalid input. Please enter a numeric value.")
```

```
vowel count.py  voting eligibility.py  ATM Machine.py
C: > Users > hi > OneDrive > Documents > AI assistant coding > ATM Machine.py > ...
60 def atm_menu():
61     print("Please enter a positive amount.")
62     try:
63         amount = float(input("Enter amount to deposit: $"))
64         if 0 < amount <= balance:
65             balance += amount
66             print(f"${amount:.2f} deposited successfully.")
67         elif amount > balance:
68             print("Insufficient funds.")
69         else:
70             print("Please enter a positive amount.")
71     except ValueError:
72         print("Invalid input. Please enter a numeric value.")
73
74     # Withdraw Money option
75     elif choice == '3':
76         try:
77             amount = float(input("Enter amount to withdraw: $"))
78             if 0 < amount <= balance:
79                 balance -= amount
80                 print(f"${amount:.2f} withdrawn successfully.")
81             elif amount > balance:
82                 print("Insufficient funds.")
83             else:
84                 print("Please enter a positive amount.")
85         except ValueError:
86             print("Invalid input. Please enter a numeric value.")
87
88     # Exit option
89     elif choice == '4':
90         print("Thank you for using the ATM. Goodbye!")
91         break
92
93     # Invalid option handling
94     else:
95         print("Invalid selection. Please choose a valid option (1-4).")
96
97 # Run the ATM menu function
98 if __name__ == "__main__":
99     atm_menu()
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
```

OUTPUT :

```
PS C:\Users\hi> & C:/Users/hi/miniconda3/python.exe "c:/Users/hi/OneDrive/Documents/AI assistant coding/ATM Machine.py"
ATM Menu:
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Please select an option (1-4): 2
Enter amount to deposit: $30000
$30000.00 deposited successfully.

ATM Menu:
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Please select an option (1-4): 1
Your current balance is: $30000.00

ATM Menu:
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Please select an option (1-4): 3
Enter amount to withdraw: $10000
$10000.00 withdrawn successfully.

ATM Menu:
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Please select an option (1-4): 3
Enter amount to withdraw: $30000
Insufficient funds.
```

EXPLANATION :

The code was carefully examined to understand how the loop keeps the menu running until the user chooses to exit and how conditional branches handle balance inquiry, deposit, and withdrawal operations. Input validation was analysed to ensure that invalid menu selections, non-numeric inputs, negative amounts, and insufficient balance cases are handled correctly. The program structure was reviewed to identify and prevent logical errors, while clear comments and meaningful variable names were used to improve readability and maintainability.